

Facade Pattern

A **facade** is one simple front door in front of a messy set of services. Components ask the facade for an action ("place the order"); the facade knows which services to call, in what order, and what to undo when something fails. This guide covers the idea, how to build and use one, where it lives, its flavours, when **not** to use it, and how to test it.

1 · THE PROBLEM

A component that has to be a project manager

Placing an order needs **four services** — cart, stock, payment, receipt — called in the **right order**, with an **undo** (release the stock) if the payment fails. If the component does all of that itself:

- it injects four services it should not care about;
- the same steps get copy-pasted into every component that checks out;
- fixing the order or the rollback means editing every copy;
- testing the component means faking four services.

Component → CartApi → StockApi → PaymentApi → ReceiptApi

The component has become a **project manager**: it knows the whole process instead of just showing a button.

2 · THE IDEA

Ask the waiter, not the kitchen

In a restaurant you do not walk into the kitchen to tell the chef, the barista and the cashier what to do. You tell the **waiter** "I'll have the set menu", and the waiter coordinates everyone. The **Facade pattern** (one of the classic *Gang of Four* design patterns) is that waiter:

DEFINITION

A **facade** is a class that gives a **simple, unified interface** to a complicated subsystem.

Without a facade

Component → 4 services.
4 dependencies.
The component knows the order and the undo.

With a facade

Component → **CheckoutFacade** → 4 services.
1 dependency.
The facade knows the order and the undo.

The subsystem services do **not** go away. They stay small and focused; the facade just coordinates them.

3 · THE FACADE IN CODE

Five steps to your own facade

- 1 **List the subsystems.** Which services does one user action touch? Write them in order.
- 2 **Create the class.** `@Injectable` named `XxxFacade`, one per feature.
- 3 **Inject the services** with `inject()` — the facade is the only class that knows them.
- 4 **Expose a small API:** methods named after what the user wants, and signals for state.
- 5 **Switch the components.** They inject only the facade and delete their old service calls.

3 · THE FACADE IN CODE · CONTINUED

```
@Injectable({ providedIn: 'root' })
export class CheckoutFacade {
  private readonly cart = inject(CartApi);
  private readonly stock = inject(StockApi);
  private readonly payment = inject(PaymentApi);
  private readonly receipt = inject(ReceiptApi);

  readonly status = signal<'idle' | 'working' | 'success' | 'failed'>('idle');
  readonly message = signal('');

  async placeOrder(): Promise<void> {
    this.status.set('working');
    let reservationId: string | undefined;
    try {
      const items = await this.cart.getItems();
      reservationId = await this.stock.reserve(items);
      const total = items.reduce((sum, i) => sum + i.price, 0);
      await this.payment.charge(total);
      await this.receipt.send(reservationId);
      this.message.set('Order placed!');
      this.status.set('success');
    } catch {
      if (reservationId) await this.stock.release(reservationId); // the undo lives here
      this.message.set('Order failed');
      this.status.set('failed');
    }
  }
}
```

4 · USING IT

The component gets tiny

Components inject only the facade and read its signals. They never import the subsystem services, so those can change freely behind the facade.

```
@Component({
  selector: 'app-checkout',
  template: `
    <button (click)="checkout.placeOrder()" [disabled]="checkout.status() === 'working'">
      Place order
    </button>
    <p>{{ checkout.message() }}</p>
  `,
})
export class CheckoutComponent {
  protected readonly checkout = inject(CheckoutFacade); // the only dependency
}
```

	WITHOUT A FACADE	WITH A FACADE
Services the component knows	4	1
Handler in the component	~15 lines (order + try/catch + undo)	1 line: <code>placeOrder()</code>
Fix the order or the rollback	Edit every component that copied it	Edit one class
Test the component	Fake 4 services	Fake 1 facade

5 · WHERE IT LIVES

Folder and naming

```
src/app/features/checkout/
├─ checkout.facade.ts      ← the facade
├─ checkout.component.ts   ← talks only to the facade
└─ data/
   ├─ cart.api.ts          ← subsystems
   ├─ stock.api.ts
   ├─ payment.api.ts
   └─ receipt.api.ts
```

- One facade per **feature** (checkout, products, auth) — not one for the whole app.
- Name it **XxxFacade** so everyone recognises the role.
- Provide it with `providedIn: 'root'`, or in the feature's route `providers` if its state should live only while the feature is open.

6 · TWO COMMON FLAVOURS

Which kind of facade do you need?

FLAVOUR	WHAT IT HIDES	EXAMPLE
Service facade	Several services that must be called in order	<code>CheckoutFacade</code> (this lesson)
State facade	The state library (NgRx store, selectors, actions)	<code>ProductsFacade.products()</code> and <code>.load()</code> instead of <code>store.select / dispatch</code>
API facade	Several HTTP endpoints behind one friendly method	<code>ProfileFacade.load()</code> calls <code>/user</code> , <code>/settings</code> and <code>/avatar</code> together

7 · WHEN TO USE IT — AND WHEN NOT TO

Use it when

- One action needs **several services** in a fixed order.
- The same steps appear in **more than one component**.
- You want components that only **show and click**.
- You want to change a subsystem without touching the UI.
- A feature is hard to test because of its many dependencies.

Skip it when

- The component uses **one** service with one call — the facade would be a pass-through.
- It would grow into a **"god class"** for the whole app.
- You are putting **UI work** (templates, DOM) in it.
- Components genuinely need full control of a subsystem.

8 · NOT TO BE CONFUSED WITH

Facade vs. its look-alikes

PATTERN	PURPOSE	IN ONE LINE
Facade	Simplify a complicated subsystem	One easy front door to many classes.
Adapter	Make an incompatible interface fit	Converts one interface into the one you expect.
Mediator	Let components talk without knowing each other	A traffic controller between peers; a facade only talks one way (down).
Plain service	Hold one piece of logic or data	A single worker; a facade coordinates several workers.

9 · TESTING

Two small kinds of test

```
// 1. Test the facade: fake the subsystems, check the sequence and the rollback.
TestBed.configureTestingModule({
  providers: [{ provide: PaymentApi, useValue: { charge: () => Promise.reject(new Error('declined')) } }],
});
const facade = TestBed.inject(CheckoutFacade);
await facade.placeOrder();
expect(facade.status()).toBe('failed');    // and stock.release() was called

// 2. Test the component: fake ONLY the facade.
TestBed.configureTestingModule({
  providers: [{ provide: CheckoutFacade, useValue: { placeOrder: jasmine.createSpy(), status:
signal('idle'), message: signal('') } }],
});
```

10 · COMMON MISTAKES

Things that trip people up

- **A pass-through facade.** A class that only forwards one call adds a layer and no value.
- **The god facade.** One facade for the whole app becomes a dumping ground. Keep one per feature.
- **UI logic in the facade.** Opening dialogs, reading the DOM or building templates belongs to components.
- **Leaking the subsystems.** If the facade returns raw service objects, components start depending on them again. Return simple values and signals.
- **Forgetting error handling.** The facade is the best place for the undo/rollback and for turning errors into a message the UI can show.
- **Hiding things components must control.** If callers truly need fine-grained access, let them inject the service directly.

11 · CHEAT SHEET & QUICK QUIZ

I WANT TO...	DO THIS
Call several services for one user action	Put the sequence in a <code>XxxFacade</code> method
Keep components small	Components inject only the facade
Undo a half-finished action	Put the rollback in the facade's <code>catch</code>
Show progress / errors in the UI	Expose <code>status</code> / <code>message</code> signals
Test the feature	Fake subsystems for the facade test; fake the facade for the component test

Q: What problem does a facade solve?

A component would have to know several services, their order and the undo. The facade owns that knowledge.

Q: Does a facade replace the services?

No. It sits in front of them and calls them.

Q: Facade vs adapter?

An adapter converts one interface into another that already exists; a facade creates a new, simpler interface over many classes.

Try it live: youssefezzat17.github.io/angular-lab/facade — place the same order with and without a facade, and decline the card to see the rollback.